p1227R0
Signed size() functions

Published Proposal, 2018-10-08

This version:
http://wg21.link/p1227r0

Author:
Jorg Brown (Google)

Audience:
LEWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper proposes new methods and a function for accessing container sizes as signed values.

Table of Contents

§ 1. Change history

This is the initial version, [P1227R0]

§ 2. Signed size() functions: ssize() for all!

§ 2.1. Introduction

This is a proposal to add non-member std::ssize and member ssize() functions. The inclusion of these would make certain code much more straightforward and allow for the avoidance of unwanted unsigned-ness in size computations.

§ 2.2. Motivation and scope

Consider the following code:

```
template<typename T>
bool has_repeated_values(const T& container) {
  for (int i = 0; i < container.size() - 1; ++i) {
    if (container[i] == container[i + 1]) return true;
  }
  return false;
}
```

An experienced C++ programmer would immediately see the problem here, but programmers new to the language often make the mistake seen here: subtraction of 1 from a size of zero does not produce a negative value, but rather, produces a very very large positive value. So when this routine is called on an empty container, undefined behavior results.

This has been discussed ad infinitum, most recently at the standards level at the Rapperswil 2018 LEWG meeting, during a discussion of spans and ranges; the suggestion was made to put forth a proposal to explicitly add ssize() member functions to all STL containers, and to add a non-member std::ssize() function. Once these exist, programmers can simply write:

```
template<typename T>
bool has_repeated_values(const T& container) {
  for (ptrdiff_t i = 0; i < container.ssize() - 1; ++i) {
    if (container[i] == container[i + 1]) return true;
  }
  return false;
}
```

§ 2.3. One more thing - ssize_t

There’s another subtle bug in the original code: if the container’s size is larger than 2^31, the "i" index variable will overflow, resulting in UB. One might think the answer is to use size_t instead of int, but thanks to C++'s integer promotion rules, that would result in the same bug as before. One might think to use ssize_t, but it turns out that ssize_t isn’t a C or C++ concept, but rather its definition comes from unistd.h. Worse yet, for at least one version of Visual Studio, ssize_t isn’t actually signed.

Finally, it might seem to be a good idea to introduce a std::ssize_t type. This proposal opts not to do so, on the principle that API expansion is best avoided when possible, and std::ptrdiff_t fills that need already. (ptrdiff_t is defined as "an implementation-defined signed integer type that can hold the difference of two subscripts in an array object")

§ 3. Impact on the Standard

This proposal is a pure library extension that could be implemented in C++11.

§ 4. Proposed Wording

Modify the section 27.3 Header synopsis [iterator.synopsis] by adding the following near the declarations for size():

```
template <class C>
constexpr ptrdiff_t ssize(const C& c);

template <class T, ptrdiff_t N>
constexpr ptrdiff_t ssize(const T (&array)[N]) noexcept;
```

Modify the section 27.8 Container access by adding the following near the definitions for size():

```
template <class C>
constexpr ptrdiff_t ssize(const C& c);
  // Returns: static_cast<ptrdiff_t>(c.size())

template <class T, ptrdiff_t N>
constexpr ptrdiff_t ssize(const T (&array)[N]) noexcept;
  // Returns: N.
```

Modify the synopsis of all STL containers with a size() function, to include an ssize() function, which returns exactly the same value, but with a ptrdiff_t type.

Note that the code does not just return the std::make_signed variant of the container's size() method, because it's conveivable that a container might choose to represent its size as a uint16_t, supporting up to 65,535 elements, and it would be a disaster for std::ssize() to turn a size of 60,000 into a size of -5,536.

§ 5. Acknowledgements

Thanks to Riccardo Marcangelo, whose [N4017] proposal contains verbiage too good not to copy.

§ References

§ Informative References

[N4017]

Riccardo Marcangelo. Non-member size() and more. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4017.htm

[P1227R0]

Jorg Brown. Signed size() functions. URL: http://wg21.link/p1227r0